

LOCAL DEV TO RENDER DEPLOYMENT

AI Engineer Ready · Week 7 — Same Image, Different Environment

GOLDEN RULE Read \$PORT and secrets from the environment, never hardcode — that's what makes local Docker and Render the same image.

1. Three ways to run this app

Mode	Command(s)	When to use
Without Docker	<code>uvicorn src.backend.main:app --host 0.0.0.0 --port 8000</code> <code>python -m streamlit run src/frontend/app.py</code>	Day-to-day development, fast reload
With Docker (local)	<code>docker build -t multi-document-assist .</code> <code>docker run -p 8080:8080 -e ... multi-document-assist</code>	Verifying the exact image before shipping it
Deployed (Render)	git push → Render builds from Dockerfile automatically	Sharing a live URL with users/reviewers

2. Environment variables you need

Variable	Purpose
OPENAI_API_KEY	Auth for embeddings + chat completion calls
COHERE_API_KEY	Auth for Cohere-backed steps (if used by compression/reranking)
OPENAI_MODEL	Which chat model to call — missing this causes a 400 error, not a silent default
PORT	Set by the platform (Render/Azure); Streamlit must read this, never hardcode 8080

■ Pitfall — Missing OPENAI_MODEL

A real error hit while building this project: forgetting to set OPENAI_MODEL produces a 400 error at request time, not at startup — it looks like a broken API key at first glance. Check env vars are complete before debugging the API client.

3. Deploying to Render

- Push the Dockerfile-based project to a GitHub repo Render can access
- Create a new Render Web Service pointing at that repo — Render detects and builds the Dockerfile
- Set OPENAI_API_KEY, COHERE_API_KEY, OPENAI_MODEL as Render environment variables (never commit them)
- Render assigns and injects \$PORT automatically — this is exactly why the app must read it, not hardcode it
- Every subsequent git push rebuilds and redeploys automatically

4. Local Docker vs. deployed — what's identical, what differs

	Identical	Differs
Image	Same Dockerfile, same layers, same entrypoint.sh	—
Config	—	Which port is routed, where secrets come from
Debugging	Container logs show the same health-gate + startup sequence	You can't docker exec into a Render container the way you can locally

✓ **Debug locally first**

Because the image is identical between local and Render, reproduce a deployed bug locally with docker run before touching Render — it's faster to iterate on and you get full log access.

5. Interview Prep

Q: Why does deployment only need env var changes and no code changes?

A: Because the app reads \$PORT and all secrets from the environment instead of hardcoding them — the same Docker image runs unmodified on any platform that supplies the right environment.

Q: How would you debug a bug that only shows up on the deployed Render app?

A: Reproduce it locally first with the same image via docker run, since the image is identical between environments — only the surrounding configuration differs, which narrows the search fast.

Q: What's the difference between a 400 error from a missing OPENAI_MODEL and a bad OPENAI_API_KEY?

A: A bad API key typically fails auth on the very first call; a missing OPENAI_MODEL still authenticates but fails when the request tries to specify a model — both surface as errors at request time, so checking env completeness first avoids chasing the wrong cause.